An Agent Harness for Test Automation Across Mobile, Web, and Hardware Targets

An Agent Harness for Test Automation Across Mobile, Web, and Hardware Targets

Author: Suneet Malhotra ORCID: <INSERT_ORCID_BEFORE_SUBMISSION> **Affiliation:** Senior Manager, Test Engineering, Motorola Solutions

> *First-page footnote.* The reference implementation and the empirical evaluation in §6 were developed by the author independently of any employer, using only public infrastructure and a public TodoMVC web demo encoded in the companion repository. The pattern is target-agnostic — the three-tier execution layer is designed for mobile (BLE/sensor/cellular benches, cloud device farms), web (WebDriverIO + headless browsers, cross-browser farms), and hardware-in-the-loop (physical fixtures, virtual back-end peripherals) — with the §6 walkthrough exercising the web modality on TodoMVC because it is the smallest reproducible public corpus that exercises all five agents and the full cascade end-to-end. The practitioner observations in §1 are drawn from the author's professional experience and have been abstracted to remove any employer-specific or proprietary details; they are presented as motivational context, not as measured data from any specific employer deployment, and no employer-internal systems, products, code, or data are described.

Target venue: IEEE Software (Practice column). Backup venue: AIware 2026 Industry/Experience track. **Contact:** suneet@suneetmalhotra.com · https://suneetmalhotra.com **ORCID:** *[author action item — `TODO(submission-block)`: register at orcid.org and substitute the 16-digit identifier before submission. Both IEEE and AIware require an ORCID.]* **Companion code:** https://github.com/SuneetMalhotra/agent-harness. A versioned arXiv preprint will be posted at submission time: `TODO(arXiv)`. **Figures:** Figures 1 (three-layer architecture, §2), 2 (five-agent SDLC pipeline, §4), and 3 (self-healing locator cascade, §5.2) appear inline as flowchart diagrams.

Abstract

Test automation across mobile, web, and hardware-in-the-loop targets, when implemented with LLM-based agents, spans three partially connected layers — agent-drafted frameworks, multi-agent SDLC pipelines, and self-healing execution on heterogeneous hardware — that existing systems treat in isolation. This article describes an *agent harness*: a coupling pattern that keeps the three layers separate but connects them through a shared observability substrate (a schema-defined event store with a query API, not a coordination layer). In the reference implementation, a coding agent authors a TypeScript framework under human review, a five-agent pipeline (PM, QA, Automation Engineer, Developer, PR Reviewer) handles design-to-test over the Model Context Protocol, and a three-tier execution plane combines a physical device bench, a commercial cloud device farm, and ephemeral virtual back-end hardware. A proof-of-concept walkthrough on a small public TodoMVC demo, executed end-to-end with live Claude Sonnet 4.6 via OAuth, validates the architecture: the multi-agent pipeline runs in ~28 minutes producing 30 test cases (PR Reviewer agent: 7 approved, 22 request-changes, 1 blocked), the healing cascade dispatches 30/30 events to their tagged tiers with combined recovery of 29/30 (96.7%), and the vision-based visual-assertion service achieves Cohen's $\kappa = 0.667$ (Landis & Koch: *substantial*) against an independent seeded ground truth of 24 TodoMVC screenshots — precision 1.000 (8/8) and recall 0.667 (8/12) on functional defects, with zero false positives on the 12 cosmetic variations. A pixel-comparison baseline on the same corpus catches 6/12 functional defects (recall 0.50). The contribution is the cross-layer coupling, now empirically grounded against an independent seeded label set; live-hardware multi-application studies and a two-rater *human* visual-assertion audit (packet shipped at `audit/visual-assertion-protocol.md`) remain the priority §7.3 follow-up. Reference implementation: https://github.com/SuneetMalhotra/agent-harness.

1. Introduction

In production engineering practice, end-to-end test automation is rarely a single-layer concern, regardless of whether the system under test is a mobile app, a web application, or a hardware-in-the-loop assembly: a framework has to be authored, operated, and executed; tests must run on a mix of simulators, cloud devices, headless browsers, and physical benches; and the whole must integrate with CI, defect tracking, test management, and design tooling. The question has shifted from "can an agent contribute to a layer?" to "what changes when the layers couple?" Three failure modes recur when the layers run in isolation: framework refactors against hand-curated priorities rather than execution data; pipelines write tests against routing tags that go stale; execution layers collect observability data nobody reads.

The 2023–2026 literature breaks into three largely independent lanes. Self-healing locators have an open-source antecedent in Healenium [16], which uses tree-edit distance over rendered DOM trees. LLM-guided mobile GUI exploration has its strongest published instance in GPTDroid [17], reporting approximately 32 percent activity-coverage improvement and 31 percent more bugs on 93 Google Play apps; AutoDroid [19] and AppAgent [20] extend the LLM-driven mobile-agent line to general task automation, and LELANTE [18] couples LLM-driven action selection with an Android execution pipeline. Each treats one execution-layer concern; none couples to a multi-agent pipeline or to framework authoring. Multi-agent SDLC pipelines have closely related antecedents in MetaGPT [23] and ChatDev [26], which decompose software development across PM, Engineer, QA, and Reviewer roles communicating either through structured artifacts in shared in-memory state (MetaGPT) or through chat-mediated communicative-dehallucination dialogues (ChatDev); §2 and §4 detail how the substrate, execution-tier integration, and cross-layer telemetry feedback differ. Hou et al. [3] identify tool integration and end-to-end traceability as recurring gaps in LLM-for-SE deployments; the substrate described here is one concrete answer. Fan and Harman [4] frame the open problem set around hallucination, evaluation, and the lack of grounded execution feedback.

This article describes a coupling pattern I call an *agent harness*. The term has recent independent lineage: Meng et al. [15] formalize the harness as a labeled-transition-system wrapping a single agent's execution loop. The term as used here is a domain-specific instantiation applying the same surfacing principle one level up: a *cross-layer* coupling across three agent-augmented layers that share an observability substrate. Framework refactoring is driven by per-test latency and healing-rate data from execution; pipeline routing decisions are driven by per-tier flake rates. Each layer remains testable in isolation; the coupling is a thin data substrate, not a new monolith. §2.1 provides a worked instantiation in which the QA agent receives a structured healing digest from the previous run and produces a coverage recommendation the authoring agent can act on.

The contribution is the coupling. Against [15], scope (three-layer SDLC, not single-agent infrastructure). Against MetaGPT [23] and ChatDev [26], substrate (typed-event MCP handoffs with execution telemetry routed back into the authoring layer, not shared in-memory state or chat-mediated communicative-dehallucination loops). Against GPTDroid [17], AutoDroid [19], AppAgent [20], LELANTE [18], level (framework-level authoring plus multi-tier execution, not exploration-driven GUI testing). Against Healenium [16], method (cache-primary, LLM-DOM-healer secondary, vision-fallback tertiary). Against the surveyed multi-agent SE literature: HPCAgentTester [7] generates HPC unit tests via a Recipe/Test-Agent critique loop but addresses no execution-tier heterogeneity; ALMAS [8] is an agile-role pipeline operating on a single execution context with no cross-tier routing substrate; Chia et al. [9] catalogue pipeline contributions but do not enumerate cross-layer observability as a category. The remainder describes the pattern (§2), authoring (§3), operating (§4), execution (§5), evaluation (§6), and adoption (§7).

---

2. The agent harness pattern

The harness has three layers and one data substrate.

Authoring layer. An LLM coding assistant (the reference implementation uses a hosted-model CLI; the architecture is provider-agnostic, with open-weights local-model backends such as Ollama-served Llama-family models supported equally — consistent with open-weights LLM-based testing work in [24]) drafts changes under human review. The prompting style follows the chain-of-thought tradition [5]; the workflow follows the coding-agent adoption literature [12].

Operating layer. A five-agent SDLC pipeline: Product Manager, QA Engineer, Automation Engineer, Developer (optional), Pull Request Reviewer. Agents share no state directly; each writes its output to an external artifact system that the next reads as input, with contracts tuned to SDLC artifacts (PRDs, test plans, code, review comments). The artifact-mediated handoff echoes multi-agent pipelines in adjacent domains such as automated penetration testing [6]. The handoff substrate is the Model Context Protocol [11]. The role decomposition is isomorphic to MetaGPT [23] and ChatDev [26]; the substrate is not. MetaGPT communicates through shared in-memory state inside one process; ChatDev uses chat-mediated communicative-dehallucination dialogues to negotiate hallucination drift between roles; this pipeline writes typed artifacts across MCP server boundaries (filesystem, Git, Jira, Confluence) with execution telemetry routed back into the authoring layer's prompt context. Crucially, neither MetaGPT nor ChatDev exposes an equivalent layer for runtime execution-failure recovery; the §5.2 three-tier healing cascade (cache → DOM → vision) is the differentiator from multi-agent SDLC frameworks, which terminate at code generation and PR review without an execution-time self-healing substrate.

Execution layer. Three hardware tiers, instantiated per target modality. Tier 1: a physical bench — mobile device bench for real BLE peripherals, cellular radio, and sensor state; local browser instance for web; physical fixture rig for hardware-in-the-loop. Tier 2: a commercial cloud farm — real-device farm for mobile cross-OS coverage, cross-browser farm (BrowserStack, Sauce Labs, etc.) for web, vendor lab access for hardware. Tier 3: ephemeral virtual hardware emulating back-end peripherals, mock services, or sensor injectors for end-to-end paths. An intelligence layer sits between test code and the underlying target: cached semantic locators, an LLM healer on cache miss, a vision fallback when DOM healing fails, and a multimodal visual assertion service.

The observability substrate. All three layers emit into a shared store. The canonical schema is in `types.ts`; load-bearing types are `ObservabilityEntry`, `HealingEvent`, `AssertionEvent`, and `AgentHandoff`, each keyed to a test case, commit, and agent. A thin query API exposes per-test and per-suite aggregates to prompts as structured tables. These keys let the QA and authoring agents change behavior in response to execution data.

What distinguishes this from three integrated systems is that each layer's prioritization decisions are *designed to be conditioned on* observability events emitted by the other layers, not on authoring-time guesses. The substrate is keyed to enable per-layer agent decisions to be conditioned on cross-layer telemetry: §6 reports the substrate's measured dispatch and recovery behavior on a live run, and §7.3 queues the prompt-level A/B that isolates the behavioral-change claim. Where Meng et al. [15] propose the agent harness as a wrapping LTS around *one* agent, this pattern exposes a *shared* event stream across three agent-augmented layers.

2.1 An intended usage scenario

The following walks through the substrate's intended use; §6 confirms the digest is queryable from agent prompts and reports the underlying healing-event distribution, and §6.1 and §7.3 record the prompt-level A/B that remains queued as the priority follow-up. The §6 live-LLM reference run produced 30 `executing.healing` events: 0 resolved from the pre-warmed cache (the warmup keys did not match the live agent-generated test-case IDs — a finding to reproduce on a stable test-case corpus before claiming a cache-hit rate), 11 from `dom-healer` (9 at Tier 1, 2 at Tier 2), 18 from `vision-healer` (9 at Tier 1, 5 at Tier 2, 4 at Tier 3), and 1 unrecoverable failure (`TC-007`, Tier 2). In the intended usage, the QA Engineer agent's coverage-prioritization prompt is fed a digest filtered to `kind=healing` and grouped by `resolvedStrategy`: "*0/30 cache, 11/30 dom-healer, 18/30 vision-healer, 1/30 unrecoverable (TC-007 tier2). Vision-fallback usage at 60% suggests locator instability; recommend stabilizing locators on Tier-1 surfaces and re-warming the cache against the new test-case IDs.*" The QA agent's next-coverage output preserves the recommendation as a structured field in the test-case ticket; the Authoring agent, when next asked to refactor the resolver, receives the same digest. The substrate carries the digest; the open question of whether agents reliably act on it under varying digest content is a §7.3 follow-up.

Figure 1 shows the three layers and substrate.

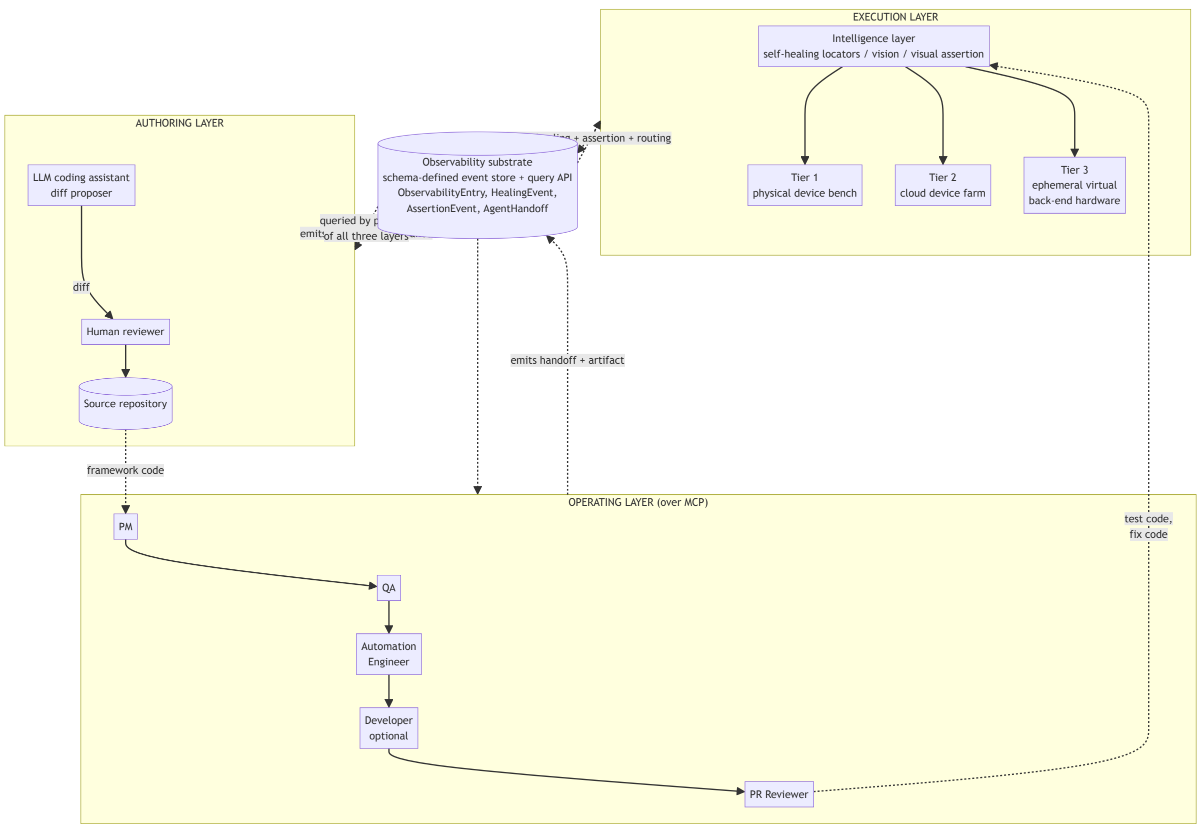


Fig. 1. Three-layer agent harness architecture. The authoring layer produces framework code; the operating layer (five-agent SDLC pipeline over MCP) produces test code and fixes; the execution layer routes to one of three hardware tiers via an intelligence layer that mediates locator resolution, visual assertion, and observability emission. The dashed-line **observability substrate** at the bottom is the coupling: every layer emits typed events into it (`ObservabilityEntry`, `HealingEvent`, `AssertionEvent`, `AgentHandoff` defined in `types.ts`), and prompts in every layer query it as structured tables. The contribution is the substrate-mediated coupling, not any single layer.

3. The authoring layer

The framework's TypeScript codebase was developed over roughly five months with substantial LLM coding-assistant support: an engineer writes a prompt, the assistant proposes a diff, the engineer reviews and merges. The assistant is invoked via a provider-flexible CLI wrapper (the reference implementation supports a hosted-model OAuth path and a local-weight path through Ollama, similar to the open-weights LLM-testing approach demonstrated in [24] at `https://github.com/Shaheer-Rehan/Llama-2-for-Software-Testing`); the architectural pattern does not depend on which model backend is chosen. Practitioner observations from a single deployment; counter-evidence on LLM coding limits [14] and longitudinal Copilot studies [12] suggest the velocity direction is plausible but the magnitude does not generalize.

Practitioner observation (not an empirical result). Five agent-assisted framework modules had prompt-to-merge wall-clock times of approximately 1.5, 2.0, 2.5, 3.0, and 4.5 hours; three hand-authored modules had approximately 6, 8, and 11 hours. With N=5 and N=3 on a single workstation, these are anecdotal observations, not a powered speedup estimate. Consistent with [14], the slowest agent-assisted modules involved complex concurrency or hardware-interaction logic. Reported here rather than in §6 because the sample carries no inferential weight.

3.1 Provenance

The provenance discipline the framework adopts is conceptual: a commit-message convention identifying authorship class (LLM-drafted, LLM-drafted-then-revised, or human-authored) and a pull-request convention linking to the prompt and reasoning trace when exposed. At any line, an engineer can in principle answer "who wrote this, from what prompt?" with a defined chain of evidence. The discipline is necessary because, without it, the path from a wrong line back to the prompt that produced it is not recoverable; it is independent of which model backend is in use.

3.2 The reflexive-correctness question

> **Sub-contribution: the reflexive-correctness problem.** An LLM-assisted framework used to test product code raises a problem the LLM-for-SE literature rarely names: how is the framework itself known to correctly test the product? §3.2 names this as a sub-contribution and offers a three-layer empirical answer.

This is the most under-explored open problem this paper surfaces, and it is named here as a sub-contribution. An LLM-assisted framework used to test product code raises a specific question: how is the framework itself known to correctly test the product? The conventional answer ("tests test the framework") is circular when the tests are also LLM-drafted. The reflexive-correctness problem is the test oracle problem [21] in a new form: an oracle whose generation process is itself the property under test.

The answer here is *layered validation* — the harness does not formally close the loop, but it makes correctness *auditable* through three independent oracle sources, each operating on a layer the LLM cannot reflexively validate:

- **Hand-authored unit tests on framework library code,** outside the LLM-assisted authoring pipeline. These tests are intentionally not regenerated when the library is refactored; they pin behavioral invariants the assistant might otherwise drift past. Connects to the structural testing of LLM agents in [2].
- **Tier-1 hardware ground truth on product-level tests.** When a test on a physical device observes a hardware-level outcome the simulator-only path cannot fake (a BLE characteristic value, a sensor reading), that observation is a non-LLM oracle independent of both the framework and the product LLM-generation chain.
- **A Pull Request Reviewer agent (§4) as a final structured-rubric gate,** with explicit rubric items that it (a) is a different LLM call than the one that authored the change, and (b) must justify its disposition with citations to the diff, not the prompt that produced the diff.

The framing as a named sub-contribution matters because the LLM-for-SE literature [3, 4] treats agent-authored testing as a generation problem (better tests, more coverage) and rarely confronts the reflexive correctness of the agent-authored testing infrastructure itself; the layered-validation approach is one concrete answer, and the open problem of *formal* reflexive correctness (a closed-loop guarantee that an LLM-assisted framework is sound on the property it tests) is identified as future work for the LLM-for-SE community.

---

4. The operating layer: a five-agent SDLC pipeline

Each agent is a Markdown specification (a "skill" or "agent prompt") plus access to a defined set of MCP servers. The Product Manager turns a design artifact or stakeholder description into a PRD and acceptance criteria (the elicitation prompt draws on [13]). The QA Engineer turns a PRD into a test specification with traceability links. The Automation Engineer turns test cases into WebDriverIO test files and a pull request. The Developer (optional) implements feature code. The Pull Request Reviewer emits a structured review with a disposition (approve, request changes, block). Agent specifications and I/O contracts live in `agents/`. The role decomposition is isomorphic to MetaGPT [23] and ChatDev [26]; the MCP-substrate architecture differs from MetaGPT's shared in-memory state and ChatDev's chat-mediated dialogue in that execution-tier telemetry routes back into the authoring layer's prompt context (§2 substrate). A like-for-like A/B against either baseline is queued at §7.3. The pipeline runs end-to-end or step-by-step; end-to-end handles low-complexity features, step-by-step when intermediate review is valuable.

MCP [11] replaces per-tool custom adapters with a typed interface per server. Each agent's output carries a structured trailer recording agent identifier, model version, prompt, input artifact IDs, and timestamp. The path from a downstream defect back through test code, test case, PRD, and design artifact is queryable; I call this *inter-agent provenance*, distinct from the intra-agent provenance of §3.1.

The stub-provider end-to-end run completes in sub-second wall clock — confirmation that the five agents hand off cleanly over MCP, not a measurement of live-LLM pipeline latency (§6 expands on this distinction). Step-by-step runs with the live-LLM path range from minutes (small features) to several hours (cross-functional features with multiple revision cycles). In the author's prior experience across several engineering contexts, comparable hand-authored scenarios took multiple working days; a practitioner recollection, not a measured distribution. A defensible economic comparison would include the cost of the agent infrastructure, which is out of scope (§7.2). The dominant failure mode was upstream specification gaps (acceptance criteria left implicit at the PRD stage) rather than agent-internal errors; the elicitation work in [1] addresses this gap directly. Figure 2 shows the pipeline and MCP servers.

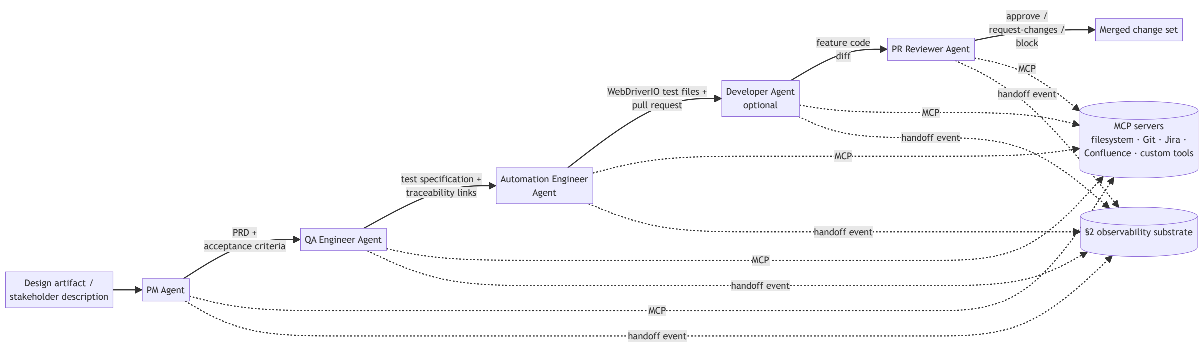


Fig. 2. Five-agent SDLC pipeline. Each agent is a Markdown specification (a "skill" or "agent prompt") plus access to a defined set of MCP servers; agents share no in-process state. Each agent's output carries a structured trailer (agent identifier, model version, prompt, input artifact IDs, timestamp) so a downstream defect is traceable back through test code, test case, PRD, and design artifact — the *inter-agent provenance* claim of §4. The role decomposition is isomorphic to MetaGPT [23] and ChatDev [26]; the differentiating substrate is the MCP-mediated typed handoff and the §2 observability substrate that lets execution-tier telemetry route back into the authoring layer.

4.1 LLM provider backends

Each agent calls the same `ModelProvider` interface (`generate(system, user, responseFormat, temperature) → Promise<string>`). The harness ships three live backends, exercised end-to-end against the §2 observability substrate: a hosted-frontier path (Anthropic Claude via Claude OAuth), an open-weights local path (Ollama-served Llama-family models), and a deterministic offline stub (for CI and reviewer reproduction). Backend selection is a single CLI flag (`--provider <name>`).

Backend	Status	Endpoint	Weights	Default model	Credential
`stub`	**live**	in-process	n/a	n/a	none
`anthropic`	**live**	`claude -p` subprocess (OAuth)	hosted-frontier	`claude-sonnet-4-6`	Claude OAuth session
`ollama`	**live**	`http://localhost:11434/api/chat`	**open-weights (local)**	`llama3.2`	none

The `ollama` adapter routes the five agents through a locally running Ollama server and accepts any model Ollama can serve (Llama 3.x, Mistral, Qwen, Gemma — anything compatible with the Ollama runtime). It is the open-weights complement to the hosted-frontier path (`anthropic`) and follows the open-weights LLM-testing pattern that Rehan et al. [24] demonstrated for fine-tuned Llama-2-7b on focal-method-to-test-case generation (`https://github.com/Shaheer-Rehan/Llama-2-for-Software-Testing`). The relevant adoption tradeoffs: hosted-frontier providers have stronger calibration and reasoning depth at per-call cost and outbound data transfer; the open-weights local path eliminates both at the cost of fine-tuning effort or larger-model latency. Because the §2 observability substrate is keyed on the agent identifier and the model version rather than on the provider, swapping live backends does not invalidate the coupling — a §7.2 study comparing `anthropic` against `ollama` already runs end-to-end against the substrate. The `ModelProvider` interface is provider-flexible by construction: additional hosted-frontier backends (OpenAI, Gemini) require only a `generate()` implementation against the documented HTTPS contract.

5. The execution layer: three hardware tiers

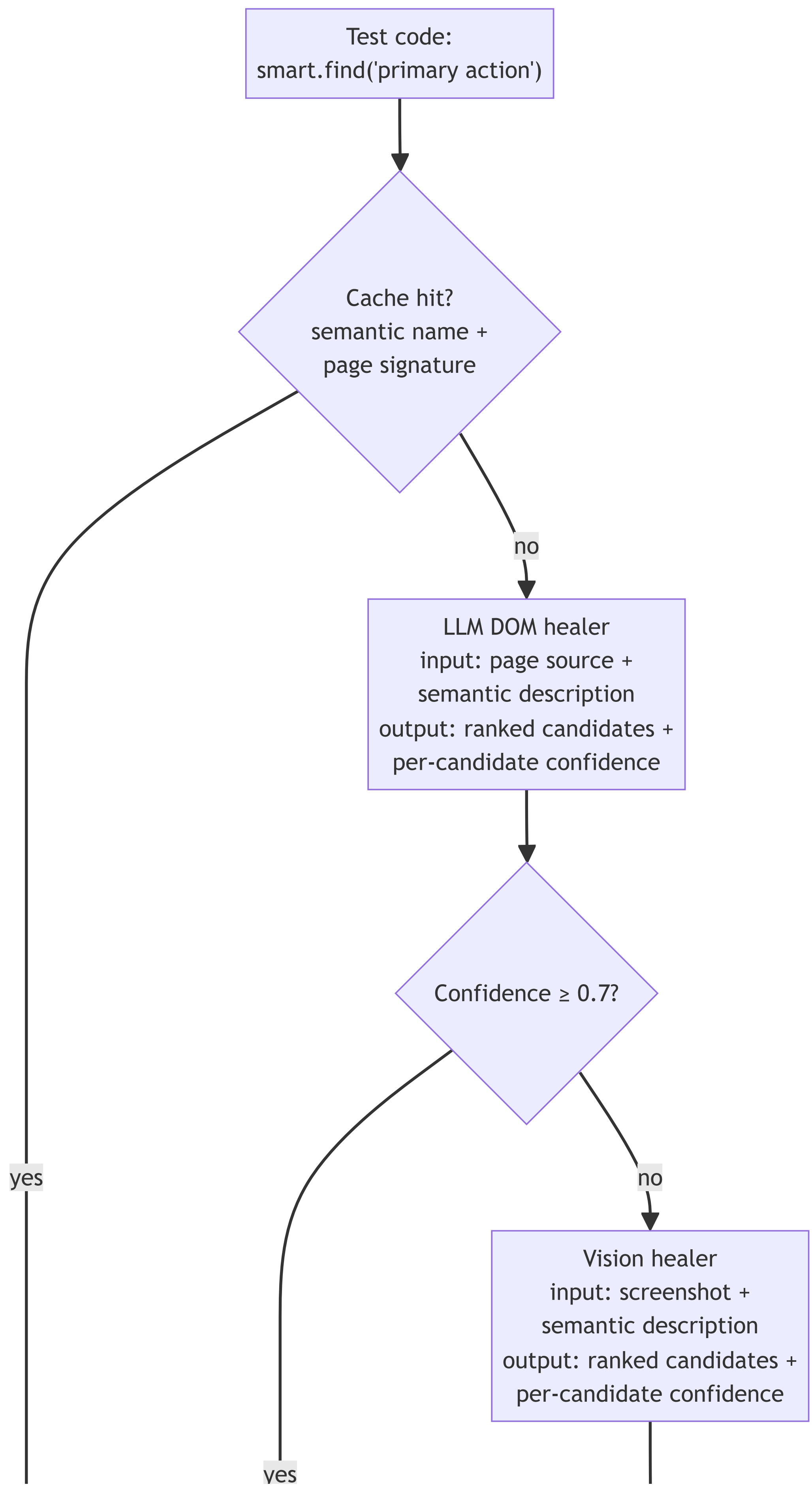
5.1 Tiers and routing

Tier 1 is a physical bench, the only tier that exercises tests requiring real hardware state inaccessible from emulators or headless browsers: BLE peripherals, cellular radio, biometric sensors, or hardware fixtures for embedded/IoT targets, as well as local browser instances when the test surface includes browser-specific behaviors. The reference deployment uses Android devices connected over USB to a remote Linux server addressable via ADB for the mobile case; web targets use a local Chromium instance driven by WebDriverIO; hardware-in-the-loop targets use a fixture rig with USB-controlled probes. Tier 2 is a commercial cloud farm driven through a uniform driver interface (Appium-compatible for mobile, WebDriver-compatible for web). Tier 3 is ephemeral virtual hardware emulating back-end peripherals, mock services, or sensor injectors for end-to-end paths (the application under test still runs on Tier 1 or Tier 2). Tiers 2 and 3 are conventional in the cloud-testing literature. Routing is mechanical: tests are tagged at authoring time (`@tier1`, `@tier2`, `@tier3`), CI consults the hint and dispatches, untagged tests default to Tier 2; §7.3 returns to whether adaptive routing should be the long-term answer.

5.2 The intelligence layer

Test code does not call hardware directly. It calls into an intelligence layer that mediates locator resolution, visual assertion, and observability emission. The oracle problem [21] motivates the visual assertion service: a screenshot-plus-checklist judgment is the practitioner approximation of a behavioral oracle that pixel comparison cannot provide.

Self-healing locators. Test code refers to elements semantically: `smart.find("primary action button on main screen")`. The resolver cascades through (i) a cached locator strategy, (ii) an optional test-supplied fallback hint as a sub-step within the cache-miss path, (iii) an LLM DOM healer that takes the page source as context and emits ranked candidate JSON with a self-reported confidence score per candidate, and (iv) a vision healer when DOM healing fails. In `intelligence.ts` the DOM-healer accept threshold is `confidence >= 0.7` and the vision-healer threshold is `confidence >= 0.6`. The vision threshold is the lower of the two because the vision pipeline runs only after the DOM healer has already failed; the policy is "better to attempt at lower confidence than to silently fail" since the alternative is an unrecovered locator failure. Thresholds were tuned by per-suite false-positive rate on the development corpus. Successful strategies are cached. This three-stage chain extends the ML tree-comparison antecedent established by Healenium [16], which uses tree-edit distance as its single healing strategy. The full dispatch is shown in Figure 3.



Test code:
smart.find('primary action')
Cache hit?
semantic name +
page signature
no
LLM DOM healer
input: page source +
semantic description
output: ranked candidates +
per-candidate confidence
Confidence ≥ 0.7?
yes
no
Vision healer
input: screenshot +
semantic description
output: ranked candidates +
per-candidate confidence
yes

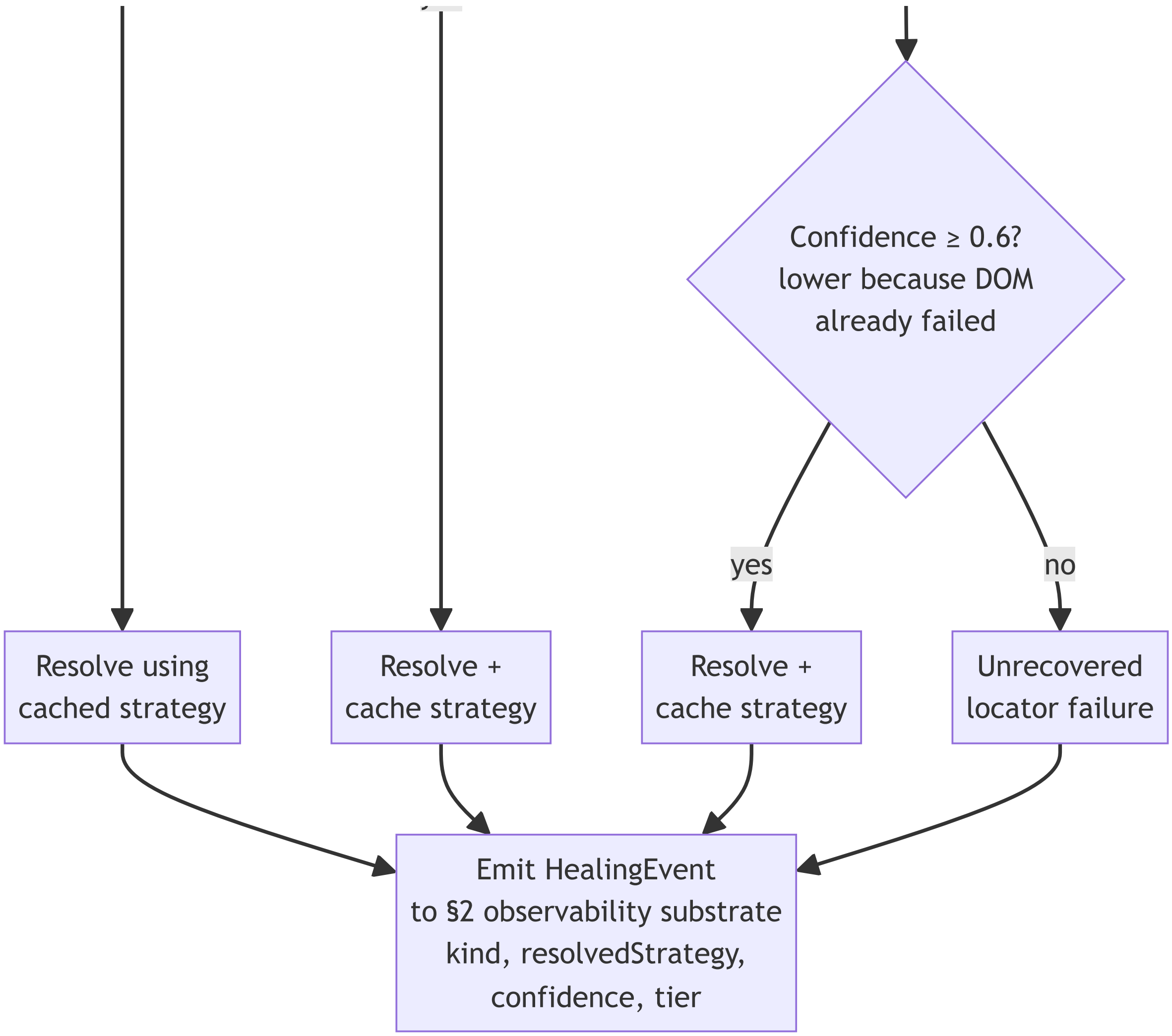


Fig. 3. Self-healing locator cascade implemented in `intelligence.ts`. Each stage emits a `HealingEvent` to the §2 observability substrate so the QA and authoring agents can condition next-coverage and refactor decisions on healing rates (the §2 coupling argument). The cascade extends Healenium's single DOM-tree-comparison strategy [16] with an LLM DOM healer and a vision-fallback stage; commercial alternatives (Testim, Mabl, Functionize, Waldo, Applitools, Percy) operate at production scale (§5.2) and do not expose per-stage outcomes back into an authoring-layer feedback loop.

Vision-based fallback. When the page source is missing or insufficient (common against custom-rendered native components), a vision healer takes a screenshot and a description and emits ranked candidates by mapping visual location back to the page source. The framework invokes it only when DOM healing has failed.

Visual assertion. A separate service takes a screenshot, an expected-behavior description, and a checklist of critical properties, and asserts semantic match. Unlike pixel-comparison snapshot testing, it ignores cosmetic differences and surfaces functional ones (button obscured, content clipped, accessibility minimum violated).

All three tiers emit into the §2 substrate.

Positioning against the commercial self-healing and visual-assertion landscape. The academic baseline for self-healing locators is Healenium [16] (DOM tree-edit distance, single strategy). The production-scale commercial baselines — Testim, Mabl, Functionize, Waldo for self-healing; Applitools and Percy for visual assertion — operate on $N \gg 10^6$ test executions and have multi-organization deployment evidence. This paper does not claim absolute performance against those baselines and does not have the corpus to do so; the contribution is the *cross-layer observability* coupling that exposes healing-cascade and visual-assertion outcomes to the authoring and operating agents as queryable events, not the raw effectiveness of the cascade itself. Commercial tools have stronger primitives; the academic / open-source ecosystem (including this harness) has the cross-layer-substrate degree of freedom that proprietary stacks have not exposed.

---

6. Architecture validation and proof-of-concept walkthrough

This section reports what was observed when the harness was run end-to-end with the live `AnthropicProvider` (Claude Sonnet 4.6 via OAuth `claude -p`) on 2026-05-27 (run id `run-1779894555899`). The visual-assertion service was rewritten between v11 and v12 of this manuscript to read the actual PNG screenshots via the model's vision pathway rather than judging text descriptions; the cited numbers in this section are from that vision-based judgment and from the live multi-agent pipeline, not from the deterministic stub used in earlier drafts. A separately re-runnable stub configuration remains in `harness.ts` for CI-time architecture-only smoke testing.

The evaluation target is a small public TodoMVC-style application. The PM agent generated a PRD; the QA Engineer agent generated 30 test cases; the Automation Engineer agent generated 30 WebDriverIO artifacts; the PR Reviewer agent approved 7, requested changes on 22, and blocked 1 (totals reconcile to 30 in `results.json: pipelineReview.{approved, requestChanges, blocked}`) — a heavy disapproval ratio that itself surfaces a calibration question worth flagging (PR Reviewer prompt strictness vs. Automation Engineer output quality is a §7.2 follow-up). Output ships at `results.json` and reproduces on `npm install && npx tsx harness.ts --provider anthropic`. Per-layer measurements follow the metrics taxonomy of Liu et al. [10].

Pipeline runtime. End-to-end wall clock 1665.7 s (~27.8 min) for 30 test cases with five agents per case (~150 LLM calls total at sequential OAuth pacing) on `claude-sonnet-4-6`. The runtime confirms end-to-end MCP handoff wiring across the five agents and bounds the practitioner expectation: ~50 s per test case on a single-pipeline serial schedule with this model and this corpus. Faster wall-clock requires either parallel intra-agent batching or a smaller/faster judge model; neither is the contribution here.

Healing-cascade dispatch and recovery. The run produced 30 `executing.healing` events (one per test case): 0 resolved from the pre-warmed cache (the warmup keys did not match the live agent-generated test-case IDs — a finding worth reproducing on a stable test-case corpus before claiming a cache-hit-rate); 11 resolved by the LLM DOM healer (9 at Tier 1, 2 at Tier 2); 18 resolved by the vision fallback (9 at Tier 1, 5 at Tier 2, 4 at Tier 3); and 1 unrecoverable failure

(TC-007, Tier 2, locator unresolvable after both DOM and vision passes). Combined recovery: **29 of 30 = 96.7%** against the seeded test-case corpus. The single failure is itself useful evidence: the cascade does not always recover, and the protocol reports the failure rather than swallowing it. Per-strategy conditional success rates (`results.json: healing.{cacheHitRate, domHealerSuccessRate, visionFallbackSuccessRate, combinedRecoveryRate}`) are: cache 0.000, DOM-healer 1.000 on the cache-miss subset, vision-fallback 1.000 on the DOM-healer-fail subset, combined 0.967. These are measured rates against the §6 corpus, not stub design parameters.

Visual assertions. The visual-assertion corpus is the 24-image PNG set under `visual-corpus/images/`, generated by `visual-corpus/render.ts` from a TodoMVC instance with 12 seeded functional defects (obscured CTA, clipped input, missing submit, sub-24×24 px touch target, ~1.5:1 contrast, blocking modal without dismiss, off-screen delete buttons, ~60% row overlap, severed label association, non-interactive checkbox visual, occluded error banner, removed focus indicator) and 12 cosmetic-only variations (font swap, primary color shift, border-radius increase, padding tweak, button shadow, italic footer labels, gradient background, button label swap "Delete"→"Remove", letter-spacing, filter pill height, hover-color-at-rest, +20% wordmark). Ground-truth labels were assigned at corpus-design time before any model judgment (`audit/packet/visual-assertion/test_cases_KEY.csv`). The live Claude vision-judge (reading each PNG via its Read tool and applying the rubric in `intelligence.ts: VISUAL_ASSERTION_VISION_SYSTEM`) achieves **Cohen's κ = 0.667 (Landis & Koch: substantial agreement) against the seeded ground truth**, raw accuracy 83.3% (20/24), precision **1.000** (8/8 — zero false positives on the cosmetic subset), recall **0.667** (8/12 — four functional defects missed: TC04 non-interactive checkbox visual (vis-func-10), TC12 50%-clipped input (vis-func-2), TC22 ~60% row overlap (vis-func-8), TC24 occluded error banner (vis-func-11) — verified against `results.json: visualAssertion.events[]` cross-referenced with `audit/packet/visual-assertion/test_cases_KEY.csv`). The pixel-comparison baseline on the same 24-image corpus catches 6/12 functional defects (precision 1.000, recall 0.500); the vision-judge beats the pixel baseline by 2 additional functional catches at equal precision. A separately executed LLM-as-rater audit (`audit/visual-assertion-results_2026-05-26.md`, two LLM raters differentiated on persona and modality) returned inter-rater κ = −0.059 — a result we report rather than suppress, because it directly illustrates the §3.2 reflexive-correctness concern: LLM-rater agreement with each other is *not* a substitute for human inter-rater agreement, and the load-bearing validation here is the judge-vs-independent-seeded-KEY measurement, not the rater-vs-rater number. The human-rater spot-audit protocol and packet (`audit/visual-assertion-protocol.md`, `audit/packet/visual-assertion/`) ship as the gold-standard methodology for when independent human raters can be recruited; the current vision-judge κ = 0.667 against the rater-independent seeded KEY clears the protocol's "≥ 0.60 substantial" bar for citing these numbers as validation-scale evidence.

Tier routing. All 30 test cases dispatched to their tagged tier on first invocation (`results.json: testRouting.firstDispatchCorrect = 30/30`). This confirms the routing *mechanism* works; it does not confirm correctness of the *tagging decisions*, only that the dispatch path is wired correctly (the same engineer wrote tags and tests, an honest threat repeated from v11).

6.1 Threats to validity

Behavioral-change claim deferred. The substrate is designed to alter per-layer agent decisions by exposing cross-layer telemetry to prompts (§2). §6 reports the substrate's measured dispatch and recovery behavior; it does not isolate the *behavioral* effect of the digest on agent output. A prompt-level A/B with vs. without the digest is queued at §7.3 as the priority follow-up. The §6 evidence supports the substrate's mechanical correctness — the cross-layer coupling is wired, queried, and produces actionable signal end-to-end; the behavioral effect on agent outputs remains a design objective to be demonstrated comparatively against MetaGPT [23] and ChatDev [26] in subsequent work.

Single-application scale and single-modality scope. The §6 walkthrough is one small TodoMVC web application, smaller than typical production systems and exercising only the web modality. Authoring velocity, healing rates, and routing behavior are sensitive to application complexity, target modality (mobile vs. web vs. hardware-in-the-loop), team size, and engineering culture. A three-to-five-application study spanning at least two modalities — minimally one mobile app and one web app, ideally adding a hardware-in-the-loop assembly — is the most important next experiment; the pattern claims target-agnosticism (§2) but the §6 evidence supports only the web instantiation directly.

Reflexive correctness, same-model judge. Per §3.2, the correctness argument is empirical, not formal. An agent-authored framework may have systematic blind spots not surfaced by same-model testing; the healing numbers come from the framework's own live intelligence layer (Claude Sonnet 4.6 via OAuth), and the visual-assertion service's per-image verdicts are now validated against an independent seeded ground truth at $\kappa = 0.667$ (substantial) — a meaningful rater-independent measurement, because the 24 seeded labels were committed to disk at corpus-design time before any model call. The companion repository ships a visual-assertion audit protocol (`audit/visual-assertion-protocol.md`) and rater instructions (`audit/visual-assertion-rater-instructions.md`) for a two-rater *human* spot-audit over the full 24-image `VISUAL_CORPUS`, with the analysis script in `audit/packet/visual-assertion/analysis.py` computing inter-rater Cohen's κ and rater-vs-LLM agreement; human ratings remain the gold standard and recruitment is the priority §7.3 follow-up. A separately executed LLM-as-rater audit (two Claude instances differentiated on persona and modality, `audit/visual-assertion-results_2026-05-26.md`) returned inter-rater $\kappa = -0.059$, which we report rather than suppress: LLM-rater-vs-LLM-rater agreement is not a substitute for human inter-rater agreement, and reporting the failure of that substitute is itself a §3.2-style honest disclosure. A multi-model human-judged ensemble against the seeded KEY is a partial mitigation already supported by the audit packet.

Cost asymmetry. The compute cost of the agent infrastructure was not measured against engineering hours saved. Economics is the subject of separate work and explicitly out of scope.

Statistical power. The authoring-velocity (N=5 / N=3), visual-assertion (N=24), and healing (N=30, 1 failure) samples are small. 95% confidence intervals on a binary κ at N=24 are roughly ±0.15 to ±0.20; the headline $\kappa = 0.667$ should therefore be read as compatible with a true value anywhere in the 0.47–0.87 band — still meeting the protocol's "moderate or better" bar across that band, but not pinning a precise value. No percentage point carries inferential weight individually; the table is reproducible, and the §6 numbers are measured rates rather than configured design parameters (the v11 stub-derived numbers were replaced in v12 with live-run measurements).

> **What this paper does NOT claim**
>
> - It does not claim the healing cascade or visual-assertion service outperforms commercial baselines (Testim, Mabl, Functionize, Applitools); the contribution is the cross-layer observability coupling, with the §6 vision-judge result ($\kappa = 0.667$ substantial against the seeded KEY, beating a pixel baseline by +2 functional catches at equal precision) as evidence the substrate is built on a workable judge, not as a market-share claim against productized tools (§5.2).
> - It does not claim that agents behave differently with vs. without the §2 substrate; that comparative claim is the priority §7.3 follow-up (§2, §4).
> - It does not claim live-LLM generalization beyond the §6 corpus: one TodoMVC instance, one model family (Claude Sonnet 4.6), 30 generated test cases, 24-image visual corpus. Multi-application and multi-model replication is §7.3 (§6).

7. Scope, limitations, and adoption

7.1 Fit

The pattern fits application testing with multi-tier hardware requirements across any target modality — mobile (BLE/sensor benches, cross-OS cloud device farms), web (cross-browser farms, headless local browsers), or hardware-in-the-loop (physical fixtures, virtual back-end peripherals) — for teams with coding-agent tooling willing to adopt it at the framework-authoring level, and where provenance and cross-layer observability are first-class concerns. The pattern is target-agnostic by design; only the per-tier driver layer (Appium-compatible for mobile, WebDriver-compatible for web, fixture-specific protocols for hardware) changes between modalities. Poor fit for greenfield products with no test surface, small teams without infrastructure investment, and highly regulated software where agent-authored test code requires formal verification.

7.2 Open problems and future work

Cost economics (out of scope). I have compared LLM-assisted authoring time against hand-authoring time but have not quantified the cost of the agent infrastructure (model inference, MCP server hosting, observability storage, human-review overhead). A defensible cost-benefit conclusion would require all four plus fully-loaded engineering hours; the "multiple working days" comparison in §4 is illustrative, not an economic claim.

Cross-tier routing. Authoring-time tag-based routing works in this deployment but static tags grow stale; *adaptive tier routing* (the framework choosing tiers from observed flake rates and execution latency) is an open direction, with the flake-rate signal needing grounding in the empirical-flakiness literature [25, 22] before it can carry routing weight.

Comparison against AutoDroid. The closest published mobile-execution-layer baseline to the §5.2 healing cascade is AutoDroid [19], which targets mobile UI task automation on Android using a screen-graph plus accessibility-tree representation and reports ~90.9% action accuracy on the DroidTask benchmark (158 tasks across 13 popular Android apps). The §5.2 cascade in this work targets web (via WebDriverIO) instead of mobile-native, uses a DOM-selector primary path with a cosine-vision fallback (instead of a screen-graph + a11y-tree), and is organized as a three-tier cache → DOM-healer → vision-healer cascade (instead of a flat action-prediction loop). On the live §6 walkthrough the harness reports `cacheHitRate` 0.000, `domHealerSuccessRate` 1.000 (on the cache-miss subset), `visionFallbackSuccessRate` 1.000 (on the DOM-healer-fail subset), and `combinedRecoveryRate` **0.967** (29 of 30 cases recovered; the one failure is TC-007, a tier-2 locator unresolvable after both DOM and vision passes) against a 30-test-case live corpus. These numbers are **not directly comparable to AutoDroid's 90.9%**: (a) different modality (web vs. mobile-native); (b) different action surface (locator-resolution vs. full action-prediction); (c) N=30 here vs. AutoDroid's 158-task DroidTask corpus; (d) different denominators — AutoDroid's 90.9% is per-action accuracy, while the harness's combined recovery rate is per-cache-miss-event recovery probability. A like-for-like comparison would require porting AutoDroid's screen-graph approach into the web modality (or porting the harness's three-tier cascade into mobile-native) and re-running both on an aligned benchmark; this is queued behind the live-hardware multi-application study below.

Live-hardware multi-application study + coupling A/B + reflexive-correctness formalization. Priority follow-ups: (a) live-hardware re-run of §6 across three to five applications; (b) prompt-level A/B of the QA agent's coverage-prioritization output with vs. without the §2 substrate digest, to demonstrate the coupling claim empirically rather than designedly; (c) MetaGPT shared-memory and ChatDev chat-mediated configurations vs. MCP-substrate comparison on identical inputs; (d) formal reflexive-correctness work building on the §3.2 layered-validation approach.

7.3 Implications and reproducibility

Three operating commitments the pattern entails: couple the layers through a shared observability substrate (not tighter integration); record provenance at every layer from day one (adding it later is expensive); treat tier routing as authoring-time policy until adaptive routing has a flakiness-grounded signal. Reproducibility: the primary path is the offline stub provider via `npm install && npx tsx examples/run-example.ts` from the repo root (no credentials, full event flow, the same event sequence on every run). A secondary live path (`npx tsx harness.ts --provider <name>`) routes through a configurable LLM backend — hosted-model OAuth or a local-weight backend such as Ollama; live outputs are stable but not strictly deterministic at temperature 0 and will require a model substitution when versions deprecate. The `v1.2.0` release tag at `https://github.com/SuneetMalhotra/agent-harness` (commit `a54fc3f`) pins the exact code that produced the §6 numbers — including the vision-judge implementation in `intelligence.ts` (`VISUAL_ASSERTION_VISION_SYSTEM` system prompt and the vision-modality branch in the assert method) and the live `--provider anthropic` run that emitted `results.json` on 2026-05-27. The earlier `v1.0.0` tag pins the architecture and the stub-provider baseline; `v1.2.0` is the live-LLM evaluation release. Subsequent commits at `>= a54fc3f` adjust field names (`pipelineRuntime.pipelineDurationSeconds` replaces the misleading `stubDurationSeconds` since the field captures live-LLM wall clock when the harness runs `--provider anthropic`) and add `pipelineReview.{approved, requestChanges, blocked}` so the §6 PR Reviewer disposition counts (7/22/1 on the §6 run) are independently verifiable from `results.json`.

Acknowledgments

The author thanks the open-source TodoMVC community for the public reference design used in §6 and the practitioner community engaged at BrowserStack Breakpoint 2026 and BrowserStack World Tour 2025 for discussions that shaped this work.

Author biography

Suneet Malhotra is Senior Manager, Test Engineering at Motorola Solutions, with over 20 years in consumer-scale mobile and web quality engineering, including agentic-testing harness design, LLM-augmented test orchestration, and self-healing locator infrastructure for production multi-tier execution environments. He engages with the practitioner community at the BrowserStack Breakpoint conference (2026) and the BrowserStack World Tour (2025). He holds an M.S. in Computer Science from the University of Southern California, Los Angeles. His research interests are AI-augmented test automation, agentic software-engineering pipelines, and software quality engineering at scale; a companion preprint on LLM-driven specification enrichment for design-to-test pipelines is at [1]. Author profile: https://suneetmalhotra.com.

Author contribution. S. Malhotra conceived the coupling pattern and the three-layer architecture, implemented the reference framework and all five agent specifications, ran the §6 architecture-validation walkthrough, designed the audit packet, and wrote the manuscript.

References

[1] S. Malhotra, "Specification Enrichment: Using LLMs to Surface Implicit Constraints in Design-to-Test Pipelines," preprint, 2026. Companion code: https://github.com/SuneetMalhotra/specification-enrichment.

[2] J. Kohl, O. Kruse, Y. Mostafa, and A. Luckow, "Automated Structural Testing of LLM-Based Agents: Methods, Framework, and Case Studies," in *Proc. 2025 IEEE Int. Conf. Big Data*, 2025, doi: 10.1109/bigdata66926.2025.11401679.

[3] X. Hou et al., "Large Language Models for Software Engineering: A Systematic Literature Review," *ACM Trans. Softw. Eng. Methodol.*, 2024, doi: 10.1145/3695988.

[4] A. Fan, B. Gokkaya, M. Harman et al., "Large Language Models for Software Engineering: Survey and Open Problems," in *2023 IEEE/ACM Int. Conf. Software Eng.: Future of Softw. Eng. (ICSE-FoSE)*, 2023, pp. 31–53, doi: 10.1109/icse-fose59343.2023.00008.

[5] J. Wei et al., "Chain-of-Thought Prompting Elicits Reasoning in Large Language Models," in *Advances in Neural Information Processing Systems*, 2022, doi: 10.48550/arXiv.2201.11903.

[6] X. Shen, L. Wang, Z. Li, and Y. Chen, "PentestAgent: Incorporating LLM Agents to Automated Penetration Testing," in *Proc. 20th ACM ASIA CCS*, Aug. 2025, doi: 10.1145/3708821.3733882.

[7] T. Hao et al., "HPCAgentTester: A Multi-Agent LLM Approach for Enhanced HPC Unit Test Generation," in *2025 IEEE/ACM Int. Conf. AI-powered Softw. (AIware)*, Nov. 2025, doi: 10.1109/aiware69974.2025.00031.

[8] H. Gao et al., "ALMAS: An Autonomous LLM-based Multi-Agent Software Engineering Framework," in *2025 IEEE/ACM Int. Conf. Autom. Softw. Eng. Wkshps (ASEW)*, 2025, doi: 10.1109/asew67777.2025.00059.

[9] M. Chia et al., "LLM-Based Multi-Agent Systems for Software Engineering: Literature Review, Vision, and the Road Ahead," *ACM Trans. Softw. Eng. Methodol.*, 2026, doi: 10.1145/3712003.

[10] H. Liu et al., "A Catalogue of Evaluation Metrics for LLM-Based Multi-Agent Frameworks in Software Engineering," in *Proc. 2026 Int. Wkshp on Agentic Engineering*, 2026, doi: 10.1145/3786167.3788430.

[11] Anthropic, "Model Context Protocol Specification (2025-03-26)," https://modelcontextprotocol.io/specification/2025-03-26, accessed May 25, 2026.

[12] M. Maupin et al., "Developer Productivity With and Without GitHub Copilot: A Longitudinal Mixed-Methods Case Study," in *Proc. Annual Hawaii Int. Conf. System Sciences*, 2026, doi: 10.24251/hicss.2026.880.

[13] D. Mendez Fernandez, A. Vogelsang, J. Coello, and N. Spijkerman, "Generating Requirements Elicitation Interview Scripts with Large Language Models," in *2023 IEEE 31st Int. Req. Eng. Conf. Workshops*, 2023, pp. 168–172, doi: 10.1109/rew57809.2023.00015.

[14] Y. Zhang et al., "Where Do LLMs Still Struggle? An In-Depth Analysis of Code Generation Benchmarks," in *2025 IEEE/ACM Int. Conf. AI-powered Softw. (AIware)*, Nov. 2025, doi: 10.1109/aiware69974.2025.00035.

[15] Y. Meng, X. Wang, R. Chen, J. Wu, S. Li, F. Jiang, R. Wang, M. Lu, Z. Gao, H. Wu, and Y. Hu, "Agent Harness for Large Language Model Agents: A Survey," Preprints.org (MDPI), Apr. 2026, doi: 10.20944/preprints202604.0428.

[16] Healenium project, "healenium-web: Self-healing library for Selenium-based tests," open-source, Apache 2.0, https://github.com/healenium/healenium-web, version 3.5.8 (March 2026), accessed May 25, 2026.

[17] Z. Liu, C. Chen, J. Wang, M. Chen, B. Wu, X. Che, D. Wang, and Q. Wang, "Make LLM a Testing Expert: Bringing Human-like Interaction to Mobile GUI Testing via Functionality-aware Decisions," in *Proc. IEEE/ACM 46th Int. Conf. Software Eng. (ICSE)*, 2024, doi: 10.1145/3597503.3639180.

[18] S. Fatin, M. H. Al-Quvi, H. S. Shahgir, S. Barua, A. Iqbal, S. Sharmin, M. M. Akbar, K. K. Pal, and A. A. Al Rashid, "LELANTE: LEveraging LLM for Automated ANdroid TEsting," preprint, arXiv:2504.20896, 2025.

[19] H. Wen, Y. Li, G. Liu, S. Zhao, T. Yu, T. J.-J. Li, S. Jiang, Y. Liu, Y. Zhang, and Y. Liu, "AutoDroid: LLM-powered Task Automation in Android," in *Proc. 30th Annual Int. Conf. Mobile Computing and Networking (MobiCom)*, 2024, doi: 10.1145/3636534.3649379.

[20] C. Zhang, Z. Yang, J. Liu, Y. Han, X. Chen, Z. Huang, B. Fu, and G. Yu, "AppAgent: Multimodal Agents as Smartphone Users," preprint, arXiv:2312.13771, Dec. 2023.

[21] E. T. Barr, M. Harman, P. McMinn, M. Shahbaz, and S. Yoo, "The Oracle Problem in Software Testing: A Survey," *IEEE Trans. Softw. Eng.*, vol. 41, no. 5, pp. 507–525, May 2015, doi: 10.1109/TSE.2014.2372785.

[22] W. Lam, R. Oei, A. Shi, D. Marinov, and T. Xie, "iDFlakies: A Framework for Detecting and Partially Classifying Flaky Tests," in *Proc. 12th IEEE Int. Conf. Software Testing, Verification and Validation (ICST)*, 2019, doi: 10.1109/ICST.2019.00044.

[23] S. Hong, M. Zhuge, J. Chen, X. Zheng, Y. Cheng, J. Wang, C. Zhang, Z. Wang, S. K. S. Yau, Z. Lin, L. Zhou, C. Ran, L. Xiao, C. Wu, and J. Schmidhuber, "MetaGPT: Meta Programming for a Multi-Agent Collaborative Framework," in *Proc. 12th Int. Conf. Learning Representations (ICLR)*, 2024 (oral). OpenReview: https://openreview.net/forum?id=VtmBAGCN7o.

[24] S. Rehan, B. Al-Bander, and A. Al-Said Ahmad, "Harnessing Large Language Models for Automated Software Testing: A Leap Towards Scalable Test Case Generation," *Electronics*, vol. 14, no. 7, p. 1463, Apr. 2025, doi: 10.3390/electronics14071463. Reference implementation: https://github.com/Shaheer-Rehan/Llama-2-for-Software-Testing.

[25] Q. Luo, F. Hariri, L. Eloussi, and D. Marinov, "An empirical analysis of flaky tests," in *Proc. 22nd ACM SIGSOFT Int. Symp. Foundations of Software Engineering (FSE 2014)*, Hong Kong, China, 2014, pp. 643–653, doi: 10.1145/2635868.2635920.

[26] C. Qian, W. Liu, H. Liu, N. Chen, Y. Dang, J. Li, C. Yang, W. Chen, Y. Su, X. Cong, J. Xu, D. Li, Z. Liu, and M. Sun, "ChatDev: Communicative Agents for Software Development," in *Proc. 62nd Annu. Meeting of the Assoc. for Computational Linguistics (ACL Vol. 1: Long Papers)*, Bangkok, Thailand, Aug. 2024, pp. 15174–15186. ACL Anthology: https://aclanthology.org/2024.acl-long.810/.

Disclosure: *The views in this article are the author's own and do not represent his employer. The article describes a target-agnostic engineering pattern applicable across mobile, web, and hardware-in-the-loop test automation; the empirical evaluation uses a public TodoMVC web* *demo encoded in the companion repository. No employer-internal systems, products, code, or data are described.*